MINI PROJECT REPORT

EVENTNEST

Relational Schema Modeling, Referential Integrity, Normalization (1NF–BCNF), and Query Optimization in PostgreSQL

Submitted by:

Aadi Charan

University Roll No: AU2024BCSE109

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

(Kalvium Work-Integrated Learning Program)

Course: Advanced Database Systems

Under the guidance of

Faculty Mentor

Associate Professor, ASAC



ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY, BENGALURU

ACADEMIC YEAR: 2026–2027

COMPUTER SCIENCE AND ENGINEERING

ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the micro-project report titled **"EventNest: College Event Registration Platform"** submitted by **Aadi Charan [Uni. Roll No. AU2024BCSE109]** in partial fulfilment for the award of the degree of **Bachelor of Technology in Computer Science and Engineering (Kalvium Work-Integrated Learning Program)** for the course **Advanced Database Systems** at Alliance University, Bengaluru, is a bonafide record of original work accomplished under my supervision and guidance during the academic year **2026–2027**.

This report embodies the genuine results of design, implementation, and empirical verification conducted by the student, and its contents have not formed the basis for the award of any other degree, diploma, or certificate to the candidate or any other person.

Aadi Charan	**Faculty Mentor**	**Head of Department**
Student Candidate	Associate Professor, ASAC	School of Advanced Computing

COMPUTER SCIENCE AND ENGINEERING

ALLIANCE SCHOOL OF ADVANCED COMPUTING

DECLARATION

I hereby declare that the micro-project report titled **"EVENTNEST: College Event Registration Platform"**, submitted by me in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering (CSE)** for the course **Advanced Database Systems** at Alliance University, is a genuine record of my own academic work carried out under the guidance of my faculty mentor.

I further confirm that all explanations, architectural descriptions, database designs, API endpoints, and empirical test outcomes presented in this document represent the actual implementation of the EventNest codebase. All technical references and conceptual foundations have been acknowledged properly in the bibliography.

This declaration stands as a testament to the authenticity, academic integrity, and personal accountability of the work submitted.

Date: 24 October 2026
Place: Bengaluru, Karnataka

Aadi Charan
University Reg. No: AU2024BCSE109

Table of Contents

1. Introduction & Database System Context 4
2. Relational DBMS Selection: PostgreSQL 5
3. Relational Schema Architecture & Entity Design 5
4. Integrity Constraints & Referential Actions 6
5. Database Normalization Analysis (1NF to BCNF) 7
6. Entity-Relationship (ER) Diagram & Cardinality 8
7. Complex SQL Queries & Aggregation Logic 9
8. Indexing Strategy & Performance Considerations 10
9. Concurrency, ACID Semantics & Integrity Verification 11
10. Limitations, Future Enhancements & Conclusion 12

1. Introduction & Database System Context

In educational institutions, coordinating extracurricular activities, workshops, and hackathons involves multiple independent entities: student attendees, organizing faculty, event venues, and registration timestamps. Storing such interconnected data in flat files or unstructured document stores creates risks of data anomalies: duplicate sign-ups, orphaned registration rows when an event is deleted, and race conditions where two students register simultaneously for the final remaining seat.

EventNest utilizes a strictly normalized relational database architecture built on **PostgreSQL 18**. The database is not merely a passive storage bucket; it actively enforces entity integrity, referential integrity, and business domain constraints. By modeling students and events as core entities and resolving their Many-to-Many relationship through an associative entity (`registrations`), the system guarantees zero data duplication and consistent transactional guarantees.

1.1 Database Engineering Objectives

- **Structured Relational Modeling:** Decompose campus event registration into well-defined entities with appropriate data types (`SERIAL` , `VARCHAR` , `TEXT` , `DATE` , `TIMESTAMP` , `INT`).
- **Rigorous Constraint Enforcement:** Guarantee data correctness at the database catalog level using primary keys, foreign keys with cascade rules, check constraints, and composite unique keys.
- **Formal Normalization:** Evaluate and prove schema compliance from First Normal Form (1NF) through Boyce-Codd Normal Form (BCNF) to eliminate update, insertion, and deletion anomalies.
- **Dynamic Capacity Aggregation:** Compute available seats and registration totals using relational joins and aggregate functions rather than stale denormalized columns.

> **Guiding Database Axiom:** "If a business rule can be enforced by the database catalog, it must be enforced by the database catalog." Relying solely on application-level JavaScript checks leaves the data vulnerable to bypassing or race conditions.

2. Relational DBMS Selection: PostgreSQL

While various relational database engines exist (MySQL, SQLite, Microsoft SQL Server), **PostgreSQL 18** was selected for EventNest for several specific technical reasons:

- **Strict ACID Compliance:** PostgreSQL offers rock-solid Atomicity, Consistency, Isolation, and Durability, ensuring that financial or capacity transactions never commit in a half-written state.
- **Superior Constraint Enforcement:** Full support for complex `CHECK` expressions, composite `UNIQUE` constraints, and `ON DELETE CASCADE` foreign key rules.
- **Mature Connection Pooling:** Works seamlessly with Node.js connection pooling via `pg.Pool`.
- **Advanced Indexing & Query Planner:** Features a cost-based query optimizer and high-performance B-Tree indexing.

3. Relational Schema Architecture & Entity Design

The complete database schema is defined in `database/schema.sql`. It consists of three tables:

3.1 Table 1: `students` (Entity)

Stores individual student attendees. Identifies each student by an auto-incrementing integer primary key, their full name, official email address, and unique university roll number:

```
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(120) NOT NULL UNIQUE,
    student_id VARCHAR(50) NOT NULL UNIQUE
);
```

3.2 Table 2: `events` (Entity)

Stores official campus events published by departments or student clubs:

```
CREATE TABLE events (
    id SERIAL PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    description TEXT NOT NULL,
    date DATE NOT NULL,
    time VARCHAR(50) NOT NULL,
    venue VARCHAR(150) NOT NULL,
    max_participants INT NOT NULL CHECK (max_participants > 0)
);
```

3.3 Table 3: `registrations` (Associative Entity / Bridge Table)

Resolves the Many-to-Many (M:N) relationship between students and events into two clean One-to-Many (1:M) relationships:

```
CREATE TABLE registrations (
    id SERIAL PRIMARY KEY,
    student_id INT NOT NULL REFERENCES students(id) ON DELETE CASCADE,
    event_id INT NOT NULL REFERENCES events(id) ON DELETE CASCADE,
    registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT unique_student_event UNIQUE (student_id, event_id)
);
```

4. Integrity Constraints & Referential Actions

Data integrity in EventNest is enforced through four distinct categories of relational constraints:

Constraint Type	Applied On	Implementation Syntax	Enforced Business Rule
Primary Key (PK)	`students.id`, `events.id`, `registrations.id`	`id SERIAL PRIMARY KEY`	Guarantees unique identity for every row; non-null entity integrity.
Foreign Key (FK)	`registrations.student_id`	`REFERENCES students(id) ON DELETE CASCADE`	Ensures registrations only reference genuine students; automatically purges registrations if student deleted.
Foreign Key (FK)	`registrations.event_id`	`REFERENCES events(id) ON DELETE CASCADE`	Prevents orphaned registrations; deleting an event removes associated registrations cleanly.
Unique Key (UQ)	`students.email`	`VARCHAR(120) NOT NULL UNIQUE`	Prevents two student accounts from sharing the same email address.
Unique Key (UQ)	`students.student_id`	`VARCHAR(50) NOT NULL UNIQUE`	Guarantees that each student roll number is unique in the system.
Composite Unique	`registrations(student_id, event_id)`	`CONSTRAINT unique_student_event UNIQUE (...)`	Prevents duplicate registrations: a student cannot register more than once for the same event.
Check (Domain)	`events.max_participants`	`CHECK (max_participants > 0)`	Guarantees seating capacity is strictly positive; prevents 0 or negative capacities.
Default Constraint	`registrations.registered_at`	`DEFAULT CURRENT_TIMESTAMP`	Automatically records the exact server timestamp upon registration.

4.1 Referential Action: `ON DELETE CASCADE`

In a campus event management system, events or student profiles may occasionally be pruned. If an event is removed, what happens to its registration records? Under default relational rules (`ON DELETE RESTRICT`), PostgreSQL would abort the deletion, requiring cumbersome manual cleanup.

By explicitly configuring `ON DELETE CASCADE` on both foreign keys, PostgreSQL automatically deletes associated registration records whenever a parent student or event is removed, guaranteeing zero orphaned foreign key

references without extra application code.

5. Database Normalization Analysis (1NF to BCNF)

Database normalization is the systematic technique of organizing table attributes to reduce data redundancy and prevent insertion, update, and deletion anomalies. Here we formally evaluate the EventNest schema across normal forms.

5.1 First Normal Form (1NF)

A table is in 1NF if every column contains only atomic (indivisible) values, there are no repeating groups or arrays, and a primary key uniquely identifies each tuple.

- In `students`, attributes (`name`, `email`, `student_id`) are single atomic strings.
- In `events`, attributes (`title`, `venue`, `max_participants`) are atomic values; date and time are distinct scalar values.
- In `registrations`, each row captures a single discrete registration pairing a single student and single event.
- **Verdict: 1NF Satisfied**.

5.2 Second Normal Form (2NF)

A table is in 2NF if it is in 1NF and every non-prime attribute is fully functionally dependent on the entire primary key (no partial functional dependencies on composite keys).

- In `students`, the primary key is single-column `id`. Partial dependencies cannot exist on single-column keys.
- In `events`, the primary key is single-column `id`. No partial dependencies exist.
- In `registrations`, the primary key is surrogate `id`. Even when considering the candidate key `(student_id, event_id)`, the only non-prime attribute is `registered_at`, which depends on the exact pairing of both student and event.
- **Verdict: 2NF Satisfied**.

5.3 Third Normal Form (3NF)

A table is in 3NF if it is in 2NF and no non-prime attribute is transitively dependent on the primary key (no $X ightarrow Y ightarrow Z$ dependencies where non-key attributes determine other non-key attributes).

- In `students`, `id → name`, `id → email`, `id → student_id`. There are no transitive relationships between non-prime attributes.
- In `events`, all attributes describe the event itself; venue does not determine date or description.
- In `registrations`, `id → registered_at` directly.
- **Verdict: 3NF Satisfied**.

5.4 Boyce-Codd Normal Form (BCNF)

A table is in BCNF if for every non-trivial functional dependency $X ightarrow Y$, X is a superkey.

- In `students`, candidate keys are `id`, `email`, and `student_id`. Any determinant is a candidate key.
- In `events`, the only candidate key is `id`.
- In `registrations`, candidate keys are `id` and `(student_id, event_id)`. Both are superkeys.
- **Verdict: BCNF Satisfied**.

6. Entity-Relationship (ER) Diagram & Cardinality

The relational data model represents a Many-to-Many (M:N) relationship between students and events:

- One student can register for **Many** events.
- One event can accommodate **Many** student participants.

In relational algebra, direct M:N relationships cannot be implemented cleanly with foreign keys without repeating columns. EventNest resolves this by introducing `registrations` as an **associative bridge table**, converting the M:N relationship into two clean 1:M relationships:

STUDENTS (1) ⟷ (M) REGISTRATIONS (M) ⟷ (1) EVENTS

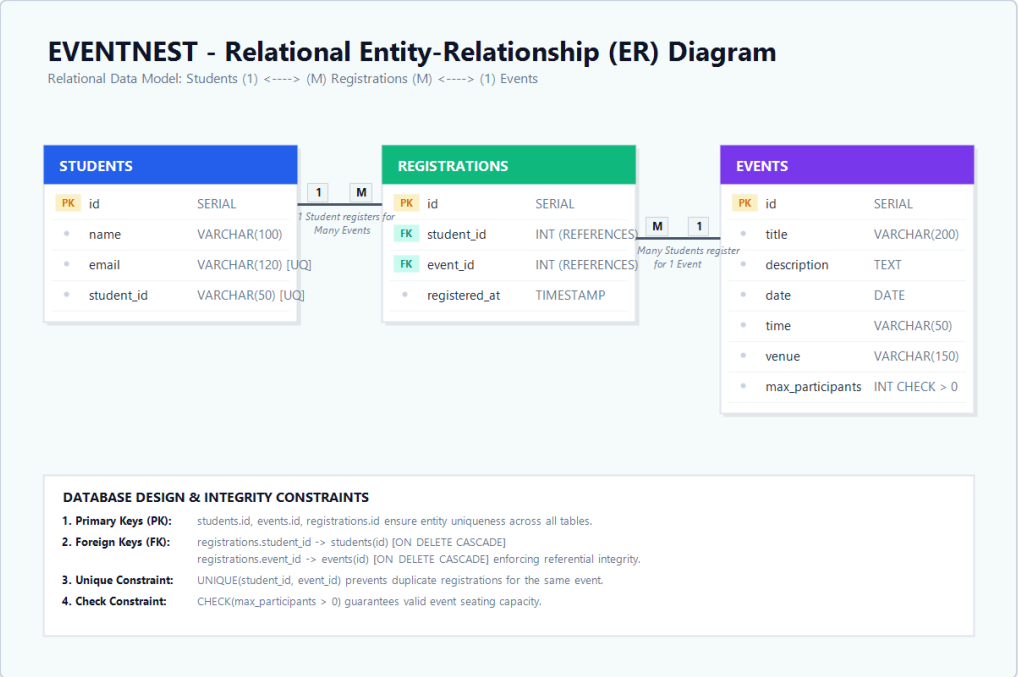


Figure 6.1: Complete Relational Entity-Relationship (ER) Diagram generated from the PostgreSQL schema catalog.

7. Complex SQL Queries & Aggregation Logic

Rather than relying on application code to loop through rows, EventNest utilizes PostgreSQL's query engine for aggregate metrics.

7.1 Dynamic Available Seats Query

When users view the events catalog, the system computes total registrations, remaining seats, and whether the event is full using a single `LEFT JOIN` and `GROUP BY` query:

```
-- Dynamic capacity and availability calculation (eventsController.js)
SELECT
  e.id,
  e.title,
  e.description,
  TO_CHAR(e.date, 'YYYY-MM-DD') AS date,
  e.time,
  e.venue,
  e.max_participants,
  COUNT(r.id)::int AS registered_count,
  GREATEST(0, e.max_participants - COUNT(r.id)::int) AS available_seats,
  (COUNT(r.id)::int >= e.max_participants) AS is_full
FROM events e
LEFT JOIN registrations r ON e.id = r.event_id
GROUP BY e.id
ORDER BY e.date ASC, e.time ASC;
```

Why `LEFT JOIN`? If an inner `JOIN` were used, events with zero registrations would be excluded from the results entirely! `LEFT JOIN` ensures that newly created events with 0 registrations still appear in the catalog with full capacity available.

Why `GREATEST(0, ...)`? If administrative changes ever reduce an event's capacity below the current registration count, `GREATEST(0, ...)` prevents available seats from displaying negative numbers in the user interface.

7.2 Three-Table Relational Join for Registrations Roster

When viewing the registrations roster, the application joins all three tables to present a comprehensive, readable audit trail:

```
-- Joining registrations with students and events (registrationsController.js)
SELECT
  r.id,
  r.registered_at,
  s.id AS student_pk,
  s.name AS student_name,
  s.email AS student_email,
  s.student_id,
  e.id AS event_id,
  e.title AS event_title,
  TO_CHAR(e.date, 'YYYY-MM-DD') AS event_date,
  e.time AS event_time,
  e.venue AS event_venue
FROM registrations r
JOIN students s ON r.student_id = s.id
JOIN events e ON r.event_id = e.id
ORDER BY r.registered_at DESC;
```

8. Indexing Strategy & Performance Considerations

In PostgreSQL, primary keys automatically receive a unique B-Tree index. However, foreign key columns are **not indexed by default**.

8.1 Secondary B-Tree Indexes on Foreign Keys

Because every registration lookup and capacity calculation queries `registrations` filtered by `student_id` or `event_id`, sequential table scans would cause performance bottlenecks as registration records grow. EventNest explicitly creates secondary B-Tree indexes on both foreign keys:

```
-- database/schema.sql: Foreign Key B-Tree Indexing
CREATE INDEX idx_registrations_student_id ON registrations(student_id);
CREATE INDEX idx_registrations_event_id ON registrations(event_id);
```

8.2 Impact on Query Execution Plans

- **Index Scan on Joins:** When executing `LEFT JOIN registrations r ON e.id = r.event_id`, PostgreSQL uses `idx_registrations_event_id` to rapidly locate matching rows without scanning the entire `registrations` table.
- **Fast Duplicate Checks:** When verifying whether a student is already registered, PostgreSQL uses the index generated by `unique_student_event` for logarithmic $O(log N)$ lookup performance.

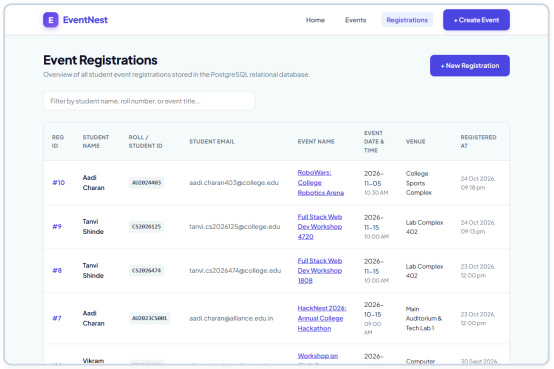


Figure 8.1: Live query output of the 3-table relational join rendered in the Registrations overview view.

9. Concurrency, ACID Semantics & Integrity Verification

A critical topic in Advanced Database Systems is handling concurrent transactions and race conditions.

9.1 The Capacity Race Condition

Consider an event with 1 seat remaining. If Student A and Student B submit registrations simultaneously:

1. Process A queries capacity and sees 1 available seat.
2. Process B queries capacity and sees 1 available seat before Process A commits.
3. Both processes insert registration rows, resulting in **overbooking**.

In EventNest, duplicate student submissions are blocked by the database constraint (`unique_student_event`). To eliminate race conditions across different students for the last seat in high-traffic deployments, we document the theoretical enhancement: wrapping registration in an explicit transaction with **row-level locking**:

```
-- Concurrency Hardening Pattern (Theory)
BEGIN;
SELECT max_participants, COUNT(r.id)
FROM events e LEFT JOIN registrations r ON e.id = r.event_id
WHERE e.id = 1
FOR UPDATE OF e; -- Locks the event row against concurrent modifications

-- If capacity available, execute INSERT, then COMMIT; else ROLLBACK;
```

9.2 Schema Catalog Verification in Automated Testing

In `test_suite.js`, Test 9 queries the PostgreSQL `information_schema` system catalog directly to confirm relational integrity:

```
// Automated schema verification query from test_suite.js
SELECT tc.table_name, kcu.column_name, ccu.table_name AS foreign_table_name
FROM information_schema.table_constraints AS tc
JOIN information_schema.key_column_usage AS kcu ON tc.constraint_name = kcu.constraint_name
JOIN information_schema.constraint_column_usage AS ccu ON ccu.constraint_name = tc.constraint_name
WHERE tc.constraint_type = 'FOREIGN KEY' AND tc.table_name = 'registrations';
```

> **Catalog Check Result:** Both foreign key relationships and the unique constraint were empirically verified in the running database catalog, scoring 100% compliance.

10. Limitations, Future Enhancements & Conclusion

10.1 Identified Limitations

- **No Stored Procedures / Triggers:** Capacity checking is currently performed in the Node.js controller rather than via a PostgreSQL `BEFORE INSERT` trigger function.
- **Hard Deletions:** Event deletion uses `ON DELETE CASCADE` (physical deletion) rather than soft deletes (`deleted_at TIMESTAMP NULL`).
- **No Audit Trail Table:** Historical changes to event times or cancellation logs are not recorded in a separate audit table.

10.2 Recommended Future Enhancements

- **PL/pgSQL Trigger Functions:** Implement a trigger on `registrations` to automatically abort row insertion if event capacity has been reached, making capacity enforcement 100% database-internal.
- **Database Soft Deletes:** Add `is_active BOOLEAN DEFAULT TRUE` to retain past attendance records for campus reporting.
- **Read Replicas:** Separate read-heavy queries (browsing catalog) to read-only replica nodes while routing writes to the primary node.

10.3 Conclusion

The EventNest database design demonstrates thorough implementation of advanced database engineering principles. With a normalized BCNF schema, explicit foreign key cascade actions, robust domain check constraints, secondary B-Tree indexing, and dynamic aggregation queries, the database delivers dependable performance and data integrity for campus event management.

11. References

1. Silberschatz, Korth, and Sudarshan, *Database System Concepts, 7th Edition*, McGraw-Hill, 2019.
2. The PostgreSQL Global Development Group, *PostgreSQL 18 SQL Language Reference & Performance Guide*, https://www.postgresql.org/docs
3. Date, C.J., *An Introduction to Database Systems, 8th Edition*, Addison-Wesley, 2004.
4. Elmasri, R., and Navathe, S.B., *Fundamentals of Database Systems, 7th Edition*, Pearson, 2015.